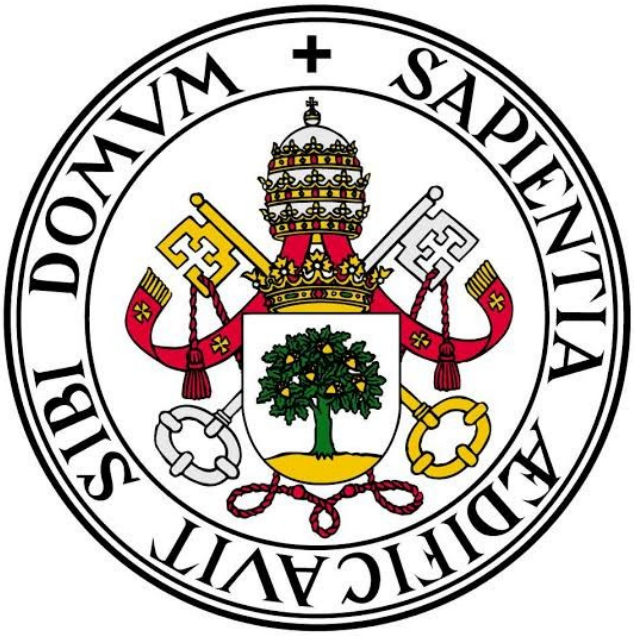
Protocolos y comunicaciones seguras: Cifrado SEAL



INGENIERÍA INFORMÁTICA

Izan Arraez Pulido

Raúl Desierio Moto

Algoritmo SEAL

El algoritmo SEAL es un algoritmo de flujo pensado para ser extremadamente rápido en software. Este se divide en 4 pasos:

En el primer paso recibe una clave de 20 bytes y con la función SHA-1 la estira formando tres tablas la T, S y R

En el segundo paso toma la posición de indice n y la combinan con la tabla R para rellenar los registros de trabajo (A, B, C, D) y los registros internos (n1, n2, n3, n4)

En el tercer paso genera el flujo de claves (Keystream), mediante 64 iteraciones mezcla los registros alterándolos con sumas matemáticas, operaciones XOR y rotaciones de bits, haciendo uso también de la tabla T para hacerlo más caótico y cuando se extraen estos datos se enmascaran con la tabla S para producir un flujo de bits pseudoaleatorios muy largo.

En el cuarto paso, se toma el flujo de bits pseudoaleatorios muy largo y se aplica la operación XOR con el mensaje original bit a bit (cifrado Vernam). Para descifrar solo hay que realizar el mismo método con el mensaje cifrado para devolver el mensaje, ya que la la operación XOR es su propio inverso.

Proceso de elaboración y distribución del trabajo

El programa se ha desarrollado con 2 sprints de 1 semana. En el primer sprint consistió en comprender el cifrado SEAL y como como nos organizaríamos y en el segundo sprint ambos implementamos el cifrado SEAL a código para luego coger las mejores partes de cada código para hacer un mejor cifrado.

Algoritmos y estrategias de programación

Algoritmos

El algoritmo se que se ha utilizado es SEAL 3.0 ya que es especialmente rápido para implementaciones software de 32 bits como se comentaba en la documentación.

Para este algoritmo se ha utilizado SHA-1, ya que necesita una función pseudoaleatoria para inicializar sus tablas. Por lo que se ha usado SHA-1 para tomar la clave maestra de 20 bytes y generar las palabras de 32 bits para rellenar las tablas T, S y R, siendo la tabla T de 512 palabras (0 al 511), la tabla S de 256 palabras (con un desplazamiento inicial en la posición 0x1000) y la tabla R de 256 palabras (con un desplazamiento inicial en la posición 0x2000) (Base + offset o desplazamiento).

Y también se a utilizado para este algoritmo el cifrado VERNAM que es el cifrado real del mensaje usando la operación XOR bit a bit

Estrategias

Como se ha usado una arquitectura de 32 bits se ha tenido que aplicar mascara (0xFFFFFFFF), ya se a simulado un procesador de 32 bits y también porque python maneja números enteros de tamaño infinito. Este se ha usado después de las sumas y de los desplazamientos de bits, para que no se produjera desbordamiento y que el algoritmo sea matemáticamente exacto.

Se han usado las librerías bitarray, secrets y hashlib. La primera se ha usado para la manipulación y concatenación de bits, la segunda para crear una clave maestra que resista ataques de predicción y la tercera para realizar el SHA-1.

Se ha optimizado la memoria mediante mascaras (0x7fc) y divisiones enteras. Esto se ha hecho para mapear los desplazamientos de bytes a los índices correctos de la tabla T, evitando errores de desbordamiento y alineando los datos a palabras de 32 bits.

Código

Para la realización del código se ha seguido 5 pasos:

1. La generación de la clave maestra

```
def crear_clave_maestra():
      return secrets.token_bytes(20)
```

2. La obtención de la palabra y la expansión de la clave

```
def obtener_palabra(clave,i):
      cont = i // 5
      resto = i % 5

      bytes_cont = cont.to_bytes(4, byteorder='big') #pasa cont a
bytes (4 bytes)

      datos_entr = clave + bytes_cont
      bloque_aleatorio = hashlib.sha1(datos_entr).digest()

      bits = bitarray()
      bits.frombytes(bloque_aleatorio)

      cortes = bits[resto*32:(resto+1)*32]

      return int.from_bytes(cortes.tobytes(), byteorder='big')

def expansion_clave(clave): #clave maestra (coge los 20 bytes)

      #Tablas de memoria precalculadas de sha-1
      T = [obtener_palabra(clave,i) for i in range(512)] # 0 a 512
      S = [obtener_palabra(clave, 0x1000 + i) for i in range(256)]
#(0x1000 es 4096)
      R = [obtener_palabra(clave, 0x2000 + i) for i in range(256)]
#(0x2000 es 8192)
return T,R,S
```

3. La inicialización del estado interno del algoritmo antes de empezar a generar el flujo de datos

```
def desp_cir(x,n):
```

```
    #Como python no tiene limete de memoria matematica esto hace
es recortar     lo que pase de 32 bits (0xFFFFFFFF)
    return ((x >> n) | (x << (32 - n))) & 0xFFFFFFFF


def initialize(n,l,R,T):

    A = n ^ R[4*l]
    B = desp_cir(n,8) ^ R[(4*l)+1]
    C = desp_cir(n,16) ^ R[(4*l)+2]
    D = desp_cir(n,24) ^ R[(4*l)+3]

    for _ in range(2):

        P = A & 0x7fc
        B = (B + T[P//4]) & 0xFFFFFFFF
        A = desp_cir(A,9)

        P = B & 0x7fc
        C = (C + T[P//4]) & 0xFFFFFFFF
        B = desp_cir(B,9)

        P = C & 0x7fc
        D = (D + T[P//4]) & 0xFFFFFFFF
        C = desp_cir(C,9)

        P = D & 0x7fc
        A = (A + T[P//4]) & 0xFFFFFFFF
        D = desp_cir(D,9)

    n1, n2, n3, n4 = D, B, A, C

    P = A & 0x7fc
    B = (B + T[P//4]) & 0xFFFFFFFF
    A = desp_cir(A,9)

    P = B & 0x7fc
    C = (C + T[P//4]) & 0xFFFFFFFF
    B = desp_cir(B,9)

    P = C & 0x7fc
    D = (D + T[P//4]) & 0xFFFFFFFF
    C = desp_cir(C,9)

    P = D & 0x7fc
    A = (A + T[P//4]) & 0xFFFFFFFF
    D = desp_cir(D,9)
```

```
    return A, B, C, D, n1, n2, n3, n4
```

4. La generación de claves

```
def generar_claves(a,n,L):

    T,R,S = a

    y = b""
    l = 0

    while(len(y) < L):

        A, B, C, D, n1, n2, n3, n4 = initialize(n,l,R,T)

        for i in range(1,65):

            P = A & 0x7fc
            B = (B + T[P//4]) & 0xFFFFFFFF
            A = desp_cir(A,9)
            B = B ^ A

            Q = B & 0x7fc
            C = C ^ T[Q//4]
            B = desp_cir(B,9)
            C = (C + B) & 0xFFFFFFFF

            P = (P+C) & 0x7fc
            D = (D + T[P//4]) & 0xFFFFFFFF
            C = desp_cir(C,9)
            D = D ^ C

            Q = (Q+D) & 0x7fc
            A = A ^ T[Q//4]
            D = desp_cir(D,9)
            A = (A + D) & 0xFFFFFFFF

            P = (P+A) & 0x7fc
            B = B ^ T[P//4]
            A = desp_cir(A,9)

            Q = (Q+B) & 0x7fc
            C = (C + T[Q//4]) & 0xFFFFFFFF
```

```
                B = desp_cir(B,9)
                P = (P+C) & 0x7fc
                D = D ^ T[P//4]
                C = desp_cir(C,9)

                Q = (Q+D) & 0x7fc
                A = (A + T[Q//4]) & 0xFFFFFFFF
                D = desp_cir(D,9)

                sal1 = (B + S[4*i-4]) & 0xFFFFFFFF
                sal2 = (C ^ S[4*i-3]) & 0xFFFFFFFF
                sal3 = (D + S[4*i-2]) & 0xFFFFFFFF
                sal4 = (A ^ S[4*i-1]) & 0xFFFFFFFF

                bits_salida = bitarray()

                bits_salida.frombytes(sal1.to_bytes(4,
byteorder='big'))
                bits_salida.frombytes(sal2.to_bytes(4,
byteorder='big'))
                bits_salida.frombytes(sal3.to_bytes(4,
byteorder='big'))
                bits_salida.frombytes(sal4.to_bytes(4,
byteorder='big'))

                y += bits_salida.tobytes()

                if len(y) >= L:
                      return y[:L]

                if i % 2 != 0:
                      A = (A + n1) & 0xFFFFFFFF
                      B = (B + n2) & 0xFFFFFFFF
                      C = (C ^ n1) & 0xFFFFFFFF
                      D = (D ^ n2) & 0xFFFFFFFF
                else:
                      A = (A + n3) & 0xFFFFFFFF
                      B = (B + n4) & 0xFFFFFFFF
                      C = (C ^ n3) & 0xFFFFFFFF
                      D = (D ^ n4) & 0xFFFFFFFF

           l += 1

     return y[:L]
```

5. El cifrado y el descifrado

```
def cifrado_descrifrado(datos,claves):

    cifrado = bitarray()

    for i in range(len(datos)):
        byte_dato = datos[i]
        byte_flujo = claves[i]

        xor = byte_dato ^ byte_flujo

        cifrado.frombytes(xor.to_bytes(1, byteorder='big'))

    return cifrado.tobytes()
```

Manual de Usuario

Como el cifrado SEAL se usa para cifrar el mensaje entre emisor y receptor el programa se ha dispuesto de esta manera.

Una vez ejecutado el programa se puede elegir entre 4 opciones, como se puede ver:

```
print("1.Enviar mensaje (cifrar)")
print("2.Recibir mensaje (descifrar)")
print("3.Mostrar variables")
print("4.Salir")
```

Antes de ejecutar la opción 1, si no se hace el envió de mensaje y se pulsa la segunda y la tercera opción el sistema mostrara un error.

```
Elige una opcion:2
Primero hay que enviar el mensaje
```

Viendo que primero hay que enviar el mensaje se selecciona la opción 1, aquí te pedirá que se escriba el mensaje y se introduzca el indice n.

```
Elige una opcion:1
Introduce un texto:Buenas tardes
Introduce posicion de cifrado (n):3
1.Fabricando clave.........
2.Expandiendo clave........
3.Generar flujo de claves..
4.Cifrado..................
```

Una vez introducido esos parámetros el programa procede a crear la clave maestra, expandir la clave, generar el flujo de claves y con todo esto a cifrar el mensaje.

Ya que se ha hecho conexión entre las dos partes, se ha simulado el envío al receptor de la clave maestra y la posición n, para que cuando se ejecute la opción 2, el programa procede a expandir la clave, a generar el flujo de claves y a descifrar el mensaje.

```
Elige una opcion:2
1.Clave maestra recibida..................
2.Indice n recibido.................
3.Expandiendo clave........
4.Generar flujo de claves..
Mensaje descifrado:Buenas tardes
```

En la tercera opción se pueden ver las variables que se han usado en hexadecimal como la clave maestra, la posición n, el cifrado, el descifrado, entre otras.

```
Clave maestra: 9f55b4525eb8321fbe4500c46f29e75a87ae4511
Flujo de claves: ba2c287cb35c15c33c67067556
Indice n: 0x00000003
Mensaje cifrado: f8594d12d22f35b75d15621025
Mensaje descifrado:4275656e617320746172646573
```

Y la opción 4 es para salir del programa

```
Elige una opcion:4
```